

ASP.NET CORE DI LIFETIMES

Singleton, Scoped and Transient – Explained with Best Practices



SINGLETON

- One instance for the entire application lifetime
- Created once and shared across all requests
- Best for stateless services and caching

Example: Cache Manager, Configuration Service



SCOPED

- One instance per request
- Created at the beginning of a request and disposed at the end of the request
- Best for database operations, repositories, business services

Example: DbContext, Repository, Business Service



TRANSIENT

- New instance every time it is requested
- Lightweight and stateless services
- Best for utilities, helpers, short-lived services

Example: Email/SMS Utility, Report Generator (stateless)



BAD EXAMPLE – Scoped service injected into Singleton

```
// Registration
services.AddScoped<IUserRepository, UserRepository>();
services.AddSingleton<NotificationService>();

public interface IUserRepository
{
    string GetUserName();
}

public class UserRepository : IUserRepository
{
    public string GetUserName() => "Animesh";
}

public class NotificationService
{
    private readonly IUserRepository _userRepository;

    public NotificationService(IUserRepository userRepository)
    {
        _userRepository = userRepository;
    }

    public void Send()
    {
        var name = _userRepository.GetUserName();
        Console.WriteLine(name);
    }
}
```

! Problem

NotificationService is Singleton (lives forever) but it depends on IUserRepository which is Scoped (per request).

After request completes, scoped service is disposed but singleton still holds reference.

Runtime Error:

Cannot consume scoped service 'IUserRepository' from singleton 'NotificationService'

CORRECT APPROACHES

Option 1 – Make both Scoped

```
services.AddScoped<IUserRepository, UserRepository>();
services.AddScoped<NotificationService>();
```

Option 2 – Use IServiceScopeFactory in Singleton

```
public class NotificationService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public NotificationService(IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    public void Send()
    {
        using var scope = _scopeFactory.CreateScope();

        var repo = scope.ServiceProvider
            .GetRequiredService<IUserRepository>();

        var name = repo.GetUserName();
        Console.WriteLine(name);
    }
}
```

- ✓ Singleton creates a temporary scope manually.
- ✓ Scoped service lives safely inside that scope.
- ✓ After completion → disposed properly.

SCOPED SERVICE – BEST EXAMPLE (DbContext)



Example Flow

- UserService, UserRepository, AuditRepository and DbContext are all Scoped.
- Same DbContext instance is shared in the whole request.
- Ensures transaction consistency and change tracking.

```
public class UserService
{
    public void CreateUser(User user)
    {
        _userRepository.AddUser(user);
        _auditRepository.AddAudit("User Created");
        _context.SaveChanges();
    }
}
```

Why Scoped is perfect here?



Same DbContext throughout request



Transaction consistency



Change tracking works properly



Better memory handling

TRANSIENT OR SCOPED FOR REPORTS?

Use Transient when:

- ✓ Stateless report generation
- ✓ Lightweight
- ✓ No DbContext / no shared data
- ✓ Independent processing

Good for:

- Excel / PDF / CSV reports
- Simple calculations
- No heavy DB work

Use Scoped when:

- ✗ Uses DbContext / repositories
- ✗ Maintains transaction
- ✗ Shares request-specific data
- ✗ Processes large datasets

Better for:

- Complex DB reports
- Multiple queries
- Enterprise scenarios



Multiple heavy reports? Focus on memory management and streaming – **lifetime** is not the main bottleneck.

COMMON REAL-WORLD USAGE

Service		Recommended Lifetime
	DbContext	Scoped
	Repository	Scoped
	Business Service	Scoped
	UnitOfWork	Scoped
	Email/SMS Utility	Transient
	Report Service (stateless)	Transient
	Report Service (uses DB)	Scoped
	Cache Manager	Singleton
	Configuration Service	Singleton

KEY TAKEAWAYS

-  Never inject Scoped service into Singleton.
-  Use Scoped for request-level data and database operations.
-  Use Transient for lightweight, stateless, independent services.
-  Choose lifetime based on responsibility, not performance.
-  Proper lifetime = fewer bugs, better scalability.

COMMON SCOPE CREATION SCENARIOS

-  BackgroundService
 -  Hangfire Jobs
 -  Worker Services
 -  Event Consumers
- Use `IServiceScopeFactory` to create scope and resolve scoped services safely.



Understand DI Lifetimes deeply to build maintainable, scalable and bug-free applications.

